ZentroMart VoiceOps 2026 — grounded voice support agent

Capstone report · LLM & Generative AI course (CAC IIT Guwahati × Gnani.ai) · dataset reference date 20 October 2026

Item	Result (all numbers produced by the saved evaluation runs)
Step 1 vs Step 2 (52 practice questions)	Full pass **16/52 (30.8%) → 51/52 (98.1%)** · same model (gemini-3.5-flash-lite), same scorer · Step 2 on ASR-like text: 47/52 (90.4%) (52/52 completed)
Multi-turn (23 conversations / 96 turns)	Turns fully passed **84/96 (87.5%)** · decisions 91/96 (94.8%) · tool calls 90/96 (93.8%) · privacy 96/96 (100.0%)
Retrieval only (no LLM rewriting, k=8)	All gold KB sections retrieved 43/45 (95.6%) (ASR text 43/45 (95.6%)) · catalog rows 13/13 (100.0%)
Voice (Gnani STT + TTS timbre-v2.5)	Live batch run: 6/7 input files handled end to end (synthetic Gnani-TTS customer clips + silence + one corrupt file)

What was built in each step

Step	What it does	Where
1 · Answer with an LLM	ZentroMart support system prompt (role, limits, AI disclosure), JSON reply + decision; no documents or tools. Used as the baseline.	agent.BaselineAgent
2 · Ground it in documents	Section-aligned chunks of all 21 KB files with front matter; multilingual embeddings (fastembed, paraphrase-multilingual-MiniLM-L12-v2) + BM25, RRF fusion; precedence prior and back-fill; catalog rows; date calculators; cited section IDs per answer.	retrieval/, agent.SupportAgent
3 · Hold a conversation	Per-session state (order, verification, attempts, tickets, language); planner resolves follow-ups; supplied lookup_order (one proof value, one retry, lock-out) and create_ticket enforced in code; ESC-1 handovers; polite refusals.	conversation.py, tools_adapter.py
4 · Give it a voice	Gnani STT (REST, ITN digits, ≤30 s clips, WAV splitting) and Gnani TTS timbre-v2.5 (Kaveri / Nalini / Poorvi); browser PCM recorder; batch runner; empty-audio recovery.	voice/, batch.py, web/

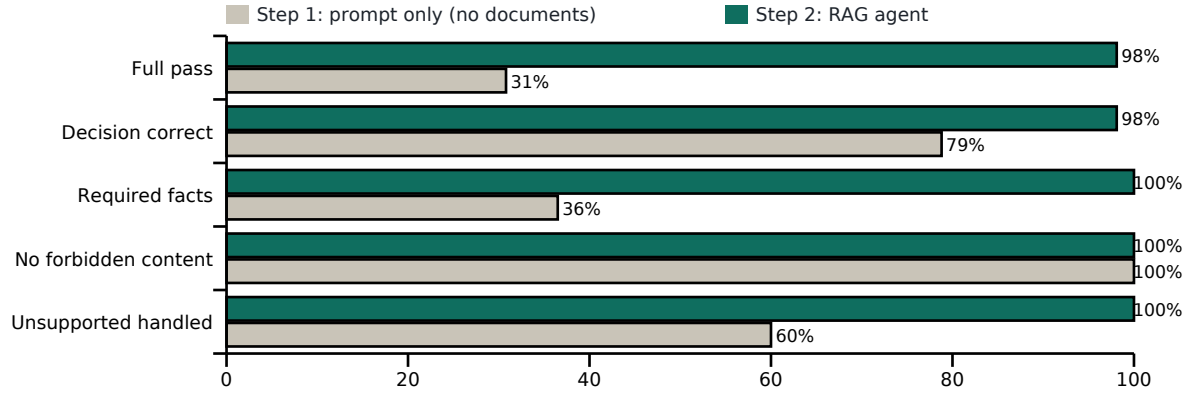
Architecture

One **SupportAgent** serves the web chat, browser voice, terminal and batch runner. Per turn: (1) a planner LLM call returns validated JSON (language, intent, order ID, proof value, English search queries with follow-ups resolved, products, dates, escalation signals); (2) code runs the identity workflow with the supplied lookup_order — digits must literally appear in the customer's message, private fields reach the model only after verified=True, the delivery pincode never; (3) code creates the ESC-1 ticket before any handover claim; (4) hybrid retrieval; (5) a responder LLM call with calculator tools and create_ticket writes the reply as validated JSON with cited IDs; (6) guardrails enforce AI disclosure, decision consistency, secret scrubbing and honest ticket failures. Gold answers are read only by the evaluator.

Retrieval and policy precedence

KB-00.1's order is metadata on every chunk (policy 1 · agent guide 2 · help 3 · FAQ 4 · marketing 5 · superseded 9). It drives an authority prior in ranking, the labels the model sees, and **back-fill**: whenever an FAQ/help/marketing chunk is retrieved the best current-policy chunk on the same topic is added (FAQ-1's generic “5 to 7 days” is always next to REF-2). pol_returns_v3_superseded.md keeps its applicability note on each chunk and is demoted unless the customer mentions a date before 1 August 2026 — then RETv3-2 (10-day electronics window) is surfaced together with RET-3, which preserves it. Agent guides and KB-00 are always in context as standing rules. Business-day and calendar arithmetic is never done by the model: it calls business_day_window / add_calendar_days, which skip Sundays and the holidays table.

Step 1 vs Step 2 — the same 52 questions, the same scorer



Metric (count / completed)	Step 1 · prompt only	Step 2 · RAG agent
Completed cases	52	52
Full pass (decision + facts + no forbidden)	16/52 (30.8%)	51/52 (98.1%)
Decision in acceptable_decisions	41/52 (78.8%)	51/52 (98.1%)
All must_include facts	19/52 (36.5%)	52/52 (100.0%)
No must_not_include phrase	52/52 (100.0%)	52/52 (100.0%)
Unsupported questions handled	3/5 (60.0%)	5/5 (100.0%)
All gold evidence cited by the answer	—	31/47 (66.0%)
Hard negative cited	—	0/52 (0.0%)

Scoring (deterministic, see evaluation/scoring.py): every ' ; ' fact of must_include is required and ' / ' alternatives are equivalent; dates match in any format (ISO, “27 October 2026”, “27th of October”, Hindi month names); numbers match as digits or number words (English/Hindi/Hinglish); any must_not_include phrase fails the case. Decision = the agent's declared decision, accepted if listed in acceptable_decisions. Step 1 has no sources, so evidence metrics apply to Step 2 only. Limitations: paraphrased forbidden claims are not detected; facts outside the listed must_include are not checked; the scorer was written against the practice set, which was also used for development.

What failed, what improved

• **Step 1** mostly fails on ZentroMart-specific facts (it does not know RET-2 windows, REF-2 timelines, the holiday table or which SKUs carry ZentroMart Assured) and sometimes answers unsupported questions with plausible inventions.
• **Iteration 1** (Step 2 43/51 (84.3%)): the agent answered general policy questions and then asked for an order ID (scored “clarify”), dropped a figure, and inferred an unsupported fact (“no store in Kochi”). Fixes: the order workflow only starts when the answer depends on the customer's own order; rules to state the evidence's figures and never deduce facts from missing information; retries for transient network errors.
• **Iteration 2** (Step 2 48/52 (92.3%), multi-turn 86/92 (93.5%) of completed turns): return answers gave the window but not the concrete last date or the ₹49 pickup fee; an overdue UPI refund was not handed over; a refusal lacked an apology; shipped-order cancellations omitted the RTO refund timing. Fixes: a deterministic refund-timeline check that reads the REF-2 table from the KB (ESC-1 rule 5 now enforced in code), cross-reference back-fill (CAN-2 → REF-1), prompt rules for concrete dates, fees, cross-references and brief apologies, and a narrower decision override.
• **Final run** (same model, same scorer, final code): Step 2 51/52 (98.1%), multi-turn 84/96 (87.5%). The only Step 2 miss (PQ0048) has the right facts but ends by asking for an order ID (scored “clarify”). Multi-turn misses: (a) a **regression** — with the new refund check and order facts, the model handed over on the verification turn of the repeat-contact / overdue-refund conversations, one turn before the customer stated the issue (6 turns: T3 early ticket, T4 ticket re-used); (b) the festive-sale rule applied to an order placed on 30 September (wrong deadline); (c) refund dates computed for a refund that had not started; (d) shipped-order cancellations labelled “answer” and missing the RTO refund timing; (e) two refusals that apologise with the Hindi “khed hai”, which the automatic scorer does not recognise (correct on manual review).

Retrieval quality (no LLM, customer text as the only query)

Metric	Typed question	ASR-like text
All gold KB sections in top-8 (+back-fill)	43/45 (95.6%)	43/45 (95.6%)
Gold KB section recall	58/60 (96.7%)	58/60 (96.7%)
MRR of first gold section	0.68	0.63
Catalog rows resolved (products.csv:SKU)	13/13 (100.0%)	13/13 (100.0%)
Questions with a hard negative in context	1/5 (20.0%)	1/5 (20.0%)

Backend: fastembed:sentence-transformers/paraphrase-multilingual-MiniLM-L12-v2 (semantic multilingual embedding). Standing sections (KB-00, STY, IDV, ESC) are always in context and excluded from recall. In the full agent, the planner's English rewrites are added as extra queries, so live retrieval is at least as good. Comparison during development: the larger paraphrase-multilingual-mpnet model did not beat MiniLM here, so the 220 MB model was kept.

Multi-turn evaluation — 23 conversations, 96 customer turns, state reset per conversation

Metric (count / turns completed)	Result
Turns completed	96
Turn full pass (decision + facts + forbidden + tools + privacy)	84/96 (87.5%)
Expected decision	91/96 (94.8%)
must_include satisfied	90/96 (93.8%)
No forbidden content	96/96 (100.0%)
Tool calls exactly as expected	90/96 (93.8%)
No private-data / secret leak	96/96 (100.0%)
First-turn AI disclosure	23/23 (100.0%)
Follow-up turns (turn ≥ 4) pass	18/27 (66.7%)
Conversations with every turn passing	14/23 (60.9%)
Turns whose gold evidence was all cited	80/96 (83.3%)

Decision accuracy by expected decision: answer: 33/36 (91.7%) · clarify: 48/48 (100.0%) · escalate: 6/6 (100.0%) · refuse: 4/6 (66.7%)
Conversations fully passed by scenario: cancel shipped order 0/2 · cancel unshipped order 2/2 · compensation demand 0/2 · express late credit 2/2 · failed verification 2/2 · high value damage 1/1 · human requested 2/2 · refund status 1/2 · repeat contact 0/2 · return eligibility 0/2 · topic switch 2/2 · where is my order 2/2
Tool-call checks compare each turn's actual calls with expected_tool_calls (order ID, presence and value of the proof, created=True for tickets) and fail on missing or unexpected calls. The privacy check (evaluator-only data) fails a turn if the reply contains the order's dates before verification, the delivery pincode or the phone suffix.

Conversation and tool handling

• Verification: order ID → lookup_order(id) (verification prompt only) → one proof value → lookup_order(id, value); one retry; after the second failure the order is locked for the session and only general help is offered. Verified records live only in that session.
• Follow-ups and topic switches are resolved by the planner from the conversation; the verified order stays available, so “when was it shipped?” after a coupon question needs no repetition.
• Handovers: human request, unauthorised activity, legal threats, safety risks, >₹10,000 damage (order item price or catalog price) and repeat contacts (contacts_last_7_days ≥ 2) create a ticket in code before the reply; overdue refunds are detected in code from the REF-2 table (latest date + more than 3 business days) and handed over with the RRN. Tickets are de-duplicated per reason; summaries contain issue, intent, order, what the customer said, checks and reason, with proof values scrubbed. A failed ticket is reported, never claimed.
• Refusals: compensation/vouchers outside DEL-4, cancelling for the customer (no such tool — guided to My Orders), other customers' data.

Voice integration and testing status

What	Status
Gnani SDK	gnani-vachana 0.8.1 installed; signatures verified in the SDK source (GnaniSTTClient.transcribe, GnaniTTSClient.synthesize, model='timbre-v2.5').
Live Gnani calls	Real STT and TTS requests succeeded with the participant's key (English, Hindi, Hinglish; silence → empty transcript handled).
Batch runner	Real run on 7 files: 6 handled end to end (incl. silence → spoken “didn't catch that”), 1 recorded as failure (deliberately corrupt file); exit code 1; outputs in sample_outputs/batch_run/.
Demo video	demo.mp4 (≤ 2 min) recorded through the real web UI: grounded answer (UPI refund dates), follow-up (card), “I don't know” (gift wrapping) and a human handover with a created ticket.
Human speech	Not tested: no human recordings were available. Customer clips were synthesised with other Gnani voices and are labelled as such.
Mocked tests	STT/TTS failures, empty audio, long-clip splitting, invalid files and web upload are covered by offline tests with fake clients (flagged mock).

Limitations and what I would fix next

• **Development on the practice set.** Prompts, the glossary and the scorer were refined while looking at practice failures, so the practice scores are optimistic; the hidden test set is the real measure.
• **Free-tier LLM.** Results use Gemini 3.5 Flash-Lite (free tier, 20 requests/day on the larger 3.6 Flash made it unusable for evaluation). A stronger model would likely improve multi-step reasoning (e.g. applying the festive-sale rule to the order date). The code supports Anthropic models via LLM_PROVIDER=anthropic without changes.
• **Scorer.** Deterministic checks cannot see paraphrased forbidden claims or partially wrong extra content. A transparent LLM-judge pass (reference answer + rubric) would be a useful secondary review.
• **Voice.** Not yet tested with real human callers, noisy audio or long multi-sentence turns; STT language is fixed per call (en-IN handles English/Hinglish, hi-IN must be chosen for Devanagari Hindi). Next: streaming STT/TTS for lower latency and automatic language selection.
• **Next fixes (found in the final run, deliberately not applied so that the reported scores match the submitted code):** offer create_ticket to the model only when the current message raises an issue (ask what the customer needs right after verification); compute concrete dates only from events present in the order record; compute return deadlines deterministically (RET-2 table + festive window) instead of leaving the date logic to the model; treat “cannot cancel a shipped order” as a refusal that always includes REF-1's refund timing.
• **One post-evaluation change:** a fuzzy product-name fallback for speech misspellings (“bruli” → Brewly kettle), used only when exact matching finds nothing. Verified offline that product matching is identical for all 200 practice texts (questions, ASR text, conversation messages); the batch run and the ASR-text evaluation were run with it.
• **Identity check.** The supplied 4-digit/pincode check is an educational simulation, not production authentication.
• **Latency.** Two LLM calls per turn plus the free-tier throttle give several seconds per voice turn; a single fused call for simple turns, caching and streaming TTS would cut this.

Integrity statement

All scores in this report were computed by the included evaluation code from saved per-case results (reports/eval/*.jsonl). No answer was typed or edited by hand; no practice answers, expected decisions or evidence labels are present in prompts or the index. Mock clients are used only in offline tests and are flagged as such in every output. Speech uses Gnani only. The demo video's customer voice is synthetic (Gnani TTS) and labelled; everything after the microphone is the real pipeline.

Reproduce

Purpose	Command
Check setup (LLM, Gnani, index)	python run.py --check
Web UI	python run.py --web
Batch audio (run command)	python run.py --audio-dir INPUT --output-dir OUTPUT
All evaluations (resumable)	python run.py --eval all
Offline tests	python -m pytest -q
Rebuild this report	python scripts/build_report.py